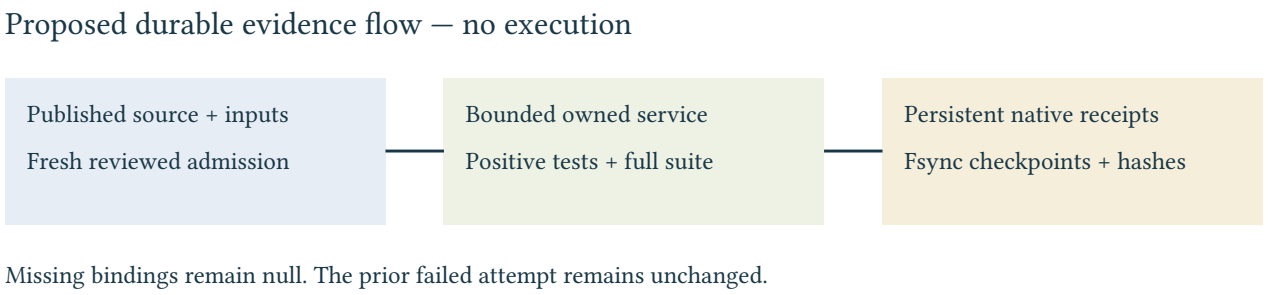
Durable validation source proposal



Durable root-library validation successor — proposed, unadmitted

T-1598 is source preparation only. It neither reopens T-1566 nor renews its exhausted attempts. No compiler, test, unit, cloud build or experiment is authorized by these files. The previous attempt remains failed with its cause unknown.

Source and scientific scope

`source-bindings.json` pins observed canonical main commit 29861c9a55f9ae5a33e1e09b3866aa397fbde605 and that revision's root Cargo.toml/Cargo.lock. The canonical pencil source has the same 481f94cc… hash as the historical candidate base. `candidate.patch` is generated against those exact main bytes and produces candidate 555a5c62…; it adds only the original cfg(test) positive control. Nothing has been applied to canonical source. A future source revision requires a new explicit binding/review, never silently substituting a new HEAD.

Keep default features and the actual Some/None solve_with_progress positive control on the unchanged first p739 fixture, seed 17823097723983109480, scalar 31124, denominator 8, coefficient cap 25 million. Keep the three existing exact guards and the full unfiltered `cargo test --release --lib --locked --offline -- --test-threads=1`. Zero tests, compile-only success, filtered tests or fake-dispatch tests do not satisfy the required full suite. This source validation does not replace the 1,260-target cohort.

Concrete source changes

`durable-worker.py` and `durable-launcher.py` are successors of the saved historical wrappers. `durable\_io.py` adds atomic JSON replacement with file and directory fsync, periodic log fsync and hashed-prefix checkpoints, and a terminal output manifest only after the supervisor verifies the owned cgroup is empty. The producer writes native receipts at startup, containment verification, before each command, after child PID capture, periodically during execution, after each command and at terminal state. Compiler/test stdout and stderr go directly to durable paths; they never require a final copy out of /var/tmp.

The proposal separates temporary workspace/cache from `/home/ec2-user/pencil-validation-receipts/<unique-attempt>/`. That persistent location must be approved, exist with private ownership, and have its actual mounted filesystem and free capacity recorded before release. Local existence alone is not a remote durability guarantee. A native checkpoint can survive a payload failure; sudden host/storage failure can still lose bytes after the last successful fsync. Running log manifests certify only observed prefixes, not a sealed complete result.

Supervisor receipts preserve argv, unit properties, InvocationID, cgroup, heartbeat identity, systemctl outputs and cleanup observations. The launcher refuses any existing attempt directory or pre-existing unit and only stops a unit with an identical captured InvocationID. If launch observation fails before ownership capture, it records unresolved cleanup and does not kill an unowned unit. Heartbeat payload fields are task_id, attempt_id, lease_id, fencing_epoch; there is no epoch alias. Fresh coordination ownership is required for execution; T-1598 is not that execution lease.

Proposed resource and retry bounds

One attempt, no automatic retries; 1,800 total wall seconds including input checks, compilation, exact tests, full suite and cleanup. The outer command is timeout –signal=TERM –kill-after=5s 1795s python3 -B durable-launcher.py. The transient unit has 2 aggregate CPUs, 16 GiB memory, no swap, TasksMax 512, OOMPolicy stop, KillMode control-group and a runtime clipped to the remaining absolute deadline. The worker also retains a 16 GiB address-space bound and single Cargo job / test thread. Those latter settings are not substitutes for cgroup enforcement.

Target cache and free-space stop thresholds remain 6 GiB; combined worker logs 256 MiB. These are sampled thresholds, not filesystem quotas, and overshoot must be retained. Supervisor metadata and durable storage must receive an explicit additional finite cap in the release review: the current source design is not admitted while this bound

is unset. The outer wall timeout also bounds receipt hashing/sealing; a killed sealing phase remains incomplete, with prior checkpoints retained. Source preparation and artifact transport costs are separately counted, never hidden as zero.

A new finite dollar ceiling, absolute UTC deadline, live CPU/RAM admission evidence and allocation are intentionally null. The expired November 4 deadline and historical USD 0.84672 estimate are not reused as authorization. Root is checking funds/live caps separately. No added infrastructure, IAM, cluster privilege or resource pool is proposed.

Required before any execution

1. Materialize a fresh private source tree from the pinned main commit, apply exactly candidate.patch, and verify the candidate source hash. Old /var/tmp paths are gone and must not be reused as if their files survived.
2. Build a new materialized-inputs.json over the complete default-feature root source and path dependencies, recursive include inputs, exact lockfile registry/vendor closure, selected cache artifacts, toolchain and system dependencies. Bind each file's resolved path, bytes and SHA-256; resolve every missing input before any compiler. Preserve source-materialization overhead. No network fallback or broad writable shared cache.
3. Bind prospective unique unit/workspace/durable paths, pinned toolchain, actual storage/cgroup controls, new valid lease fields, finite dollars/deadline and total attempt limit. Independently review worker/launcher failure paths, checkpoint/manifest scope, disk accounting and no-foreign-kill behavior. Current wrappers are source proposals and have not been syntax-checked, compiled, executed or tested.
4. Commit and push exact source, protocol/configuration, fixture/seeds, complete input manifest and launch command; verify exact GitHub revision and retain its publication receipt before any cloud build or experiment. A local commit or S3-only script is insufficient. The release's admission.json must bind that receipt and every recipe/input file; it is deliberately absent here.
5. Obtain a new explicit coordinator execution release. Publish any resulting attempt, including failures or incomplete receipts, with hashes, costs and native identifiers. A successful validation requires all exact tests, the complete unfiltered suite, unchanged inputs, and owned cleanup; missing evidence stays unknown.

   Publication status

This packet is an inspectable source proposal, not a runnable admission. The absent admission.json makes both entrypoints fail before launching any unit or compiler. All historical packets stay untouched. Graphs show the proposed evidence flow rather than unmeasured timing or cost improvement.